

Chess Engine Write-Up - Group 37

Aryan Saadati | Trent Battisti | Mani Samet

Abstract

Our project's goal is to design and train a chess engine model in which the move-selection policy is learned by a neural network, aiming to compete against the famous Stockfish chess engine at 2000 Elo. Additionally, we explore the idea of transfer learning to further improve our engine's fitting and ultimate performance. In order to measure the performance of our model(s) against the opposing engine, our evaluation technique was done based on accuracy percentage confidence intervals as defined by Lichess game analysis and the average centipawn loss confidence intervals measured using the Python chess library.

System Pipeline & Overview

Input data and pre-processing

The conventional format of chess data is in PGN (Portable Game Notation). Our streamlined data pipeline collects and generates data from the Lichess Elite Database as well as self-play games using the Stockfish engine. We then convert the data between PGN and CSV formats for cleaning and filtering, and lastly transform each game position into a 3-way tensor representation. Each tensor encodes the board state at a given move, paired with the next move played as the supervised learning target.

Base Model Architecture

Holistically, our model uses a neural network to take the pre-processed gameplay data and output a probability distribution over possible moves.

As hinted above, throughout the training process, we trained 5 different 2D Convolutional neural network models on the same data in order to test the effects of different model hyperparameters and feature engineering in the training process. Namely, we altered their respective activation functions, experimented with weighted data instead of unweighted data for training, and increased the number of both convolutional and dense layers to extract more parameters.

Finally, our base model architecture, as a system, consisted of the following: an 8x8x12 Input Layer representing the chessboard, a 64-filter 3x3 Convolution Layer with ReLU activation, a 128-filter 3x3 Convolution Layer with ReLU activation, a flattening Layer, a 256-unit Dense Layer, and a softmax output Layer.

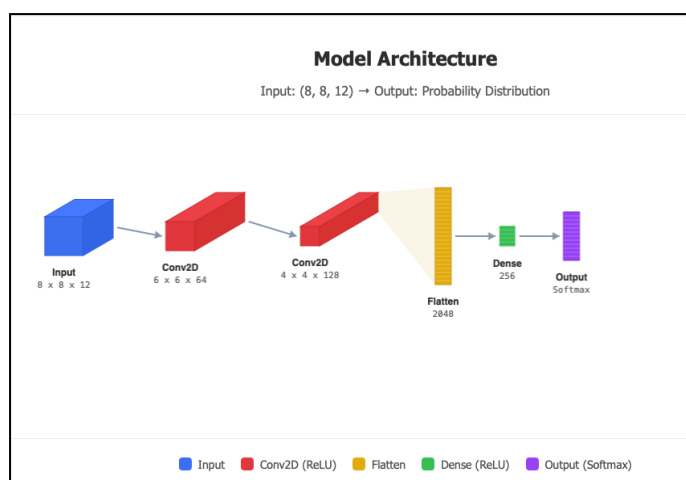


Figure 1: Base Model Architecture

Transfer Learned Model Architecture

The transfer learning technique is also used on the base model mentioned above. To do this, we freeze layers close to the input, allowing us to leave those layers' weights unchanged throughout the training. That is, our best-performing transfer learned model has its first convolutional layer frozen; instead, it has wider dense layers in order to train mostly on late-game features. Additionally, we took advantage of callback methods in order to reduce training time and avoid overfitting. Lastly, we used dropout layers to help with regularization of the big dense layers that are now trained in the model, again preventing the chance of overfitting. In terms of input data, the data used to transfer and learn this model was newly generated self-play Stockfish gameplay.

The best Transfer Learned model's neural network configuration consists of the following:

```
conv_base = tf.keras.Sequential(pretrained_model.layers[:1])

conv_base.build((None, 8, 8, 12))

conv_base.trainable = False

new_model = tf.keras.Sequential([
    conv_base,
    tf.keras.layers.Conv2D(128, (3, 3), activation='relu'),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(2048, activation='relu', name='new_dense1'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(1024, activation='relu', name='new_dense2'),
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Dense(len(move_to_int), activation='softmax', name='new_output')
])
```

Figure 2: Transfer learned model with frozen convolution Layer

**Note: We also collected a plethora of games from the Lichess dataset from July 2014. We parsed the data to only include Blitz and Classical timed controlled matches. This data can also be used to learn a model.*

Training Process

The training of all models takes the same procedure. That is, each model is trained for 50 Epochs, batch size of 64 (64 chess moves processed at a time before updating weights). Furthermore, 10% of the data is held out for validation to monitor the performance of unseen positions throughout training. Each move done is encoded using one-hot vectors so that it can be used with categorical cross entropy Loss Function, which is great for probability distribution output as it penalizes low probable moves assigned by the model.

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 2048)	60,800
new_dense1 (Dense)	(None, 2048)	4,199,352
dropout (Dropout)	(None, 2048)	0
new_dense2 (Dense)	(None, 1024)	2,806,120
dropout_1 (Dropout)	(None, 1024)	0
new_output (Dense)	(None, 1040)	1,003,600

Total params: 8,368,865 (31.93 MB)

Trainable params: 8,288,153 (31.62 MB)

Non-trainable params: 80,832 (315.75 KB)

Epoch 1/50	15743/15743	220s 14ms/step	- accuracy: 0.0549 - loss: 5.7839 - val_accuracy: 0.0888 - val_loss: 4.8647
Epoch 2/50	15743/15743	220s 14ms/step	- accuracy: 0.0812 - loss: 4.7652 - val_accuracy: 0.0929 - val_loss: 4.4754
Epoch 3/50	15743/15743	228s 14ms/step	- accuracy: 0.0920 - loss: 4.4918 - val_accuracy: 0.1001 - val_loss: 4.3258
Epoch 4/50	15743/15743	221s 14ms/step	- accuracy: 0.0997 - loss: 4.3448 - val_accuracy: 0.1023 - val_loss: 4.2371
Epoch 5/50	15743/15743	211s 13ms/step	- accuracy: 0.1060 - loss: 4.2460 - val_accuracy: 0.1057 - val_loss: 4.1867
Epoch 6/50	15743/15743	223s 14ms/step	- accuracy: 0.1110 - loss: 4.1727 - val_accuracy: 0.1082 - val_loss: 4.1479
Epoch 7/50	15743/15743	217s 14ms/step	- accuracy: 0.1154 - loss: 4.1121 - val_accuracy: 0.1083 - val_loss: 4.1221
Epoch 8/50	15743/15743	217s 14ms/step	- accuracy: 0.1198 - loss: 4.0617 - val_accuracy: 0.1100 - val_loss: 4.1012
Epoch 9/50	15743/15743	213s 14ms/step	- accuracy: 0.1234 - loss: 4.0157 - val_accuracy: 0.1105 - val_loss: 4.0870
Epoch 10/50	15743/15743	222s 14ms/step	- accuracy: 0.1272 - loss: 3.9763 - val_accuracy: 0.1117 - val_loss: 4.0718

Figure 3: Transfer Learning Model under Training

**Note: We also used another loss function called sparse-categorical cross entropy; however, we did not submit this version of our training. This required the integer encoding.*

Evaluation

The model is then evaluated against 2000 Elo Stockfish Chess Engine to play 100 games, without time control. As pointed to before, our system measures the models' performance within these 100 games based on accuracy and average centipawn loss metric.

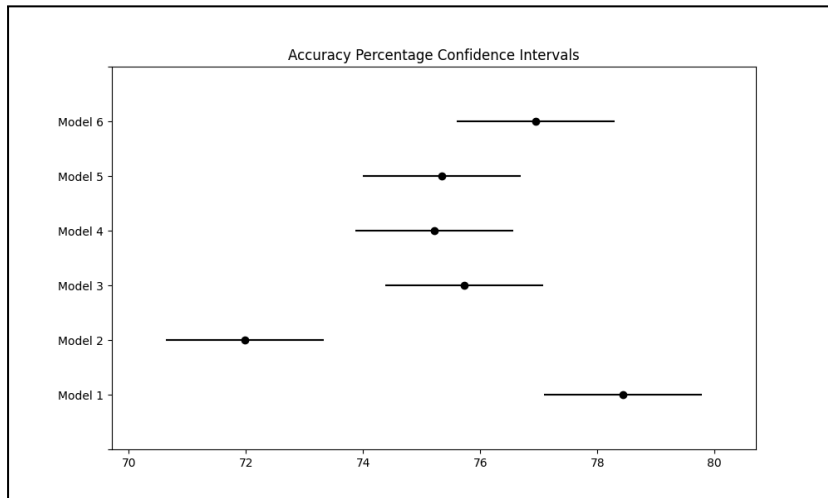


Figure 4: Average Accuracy Percentage Confidence Intervals for each model

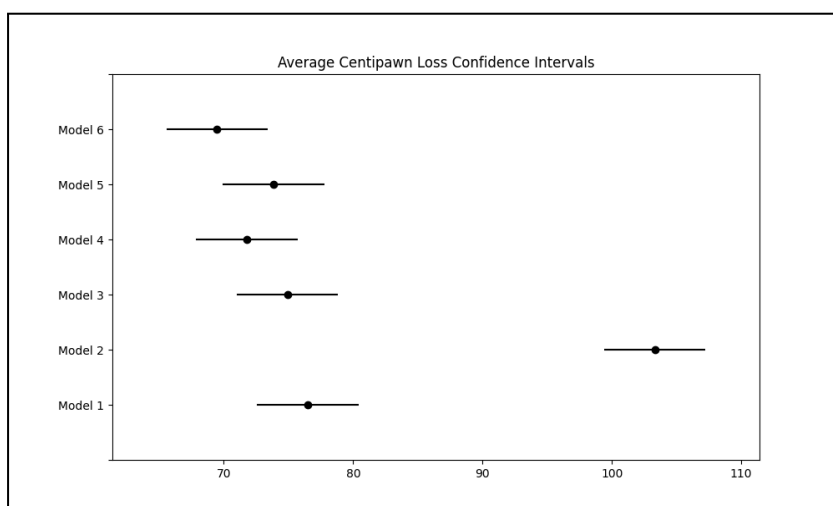


Figure 5: Average Centipawn Loss Confidence Intervals for each model

**** Model 6 represents our best performing transfer learned model and model 1 represents our base model.**

Limitations

1. From our experience with training these various models, there are many factors and variables in chess that cannot be shown with accuracy and centipawn loss. Namely, the idea of sacrificing pieces to gain an advantage in the later play cannot fit within our probability distribution as a probable win position; therefore, even a 75% accuracy or minimal blunders does not result in an ultimate win.
 - a. We understood this limitation after training different models with more than 30 hours of combined training time, and recognizing the easy possibility of overfitting for our model, concluding how complex the game of chess really is at 2000 Elo.

2. If a user wishes to use input data from time controlled human gameplay from Lichess Dataset, they should notice a significant change in playstyle as opposed to a simulated dataset. Human gameplay is much different than chess engines, and have higher average centipawn point loss than simulated gameplay; therefore, using human gameplay for transfer learning is an option that may not be beneficial for late-game improvements.
 - a. We mitigated this by filtering gameplay within longer time controlled games, where the players made less blunders due to time crunch.

Sample Gameplay

The following link also showcases the gameplay of one of our top-performing models against the Stockfish model: [▶ CMPT 310, Proof of accomplishment 3](#)

Feature Table

Description	Platform	Completeness	Code	Author(s)	Notes
Chess Data Preprocessing (Simulated)	Local	5	Python	Aryan, Mani	Fully Functional feature.
Chess Data Preprocessing (Human Games)	Local	4	Python	Aryan, Mani	Collected data from Lichess within 2000-2500 Elo. Can't choose to only use sub-samples (positions in games) in late-game.
Move Encoding (UCI → int → one-hot vectors)	Local	5	Python	Aryan, Trent	Fully functional, enables training with categorical cross-entropy Loss Function.
Simulated Models	Local	3.5	Python	Aryan, Trent, Mani	Can play generalized chess , still messes up in late-game at 2000 Elo against stockfish. More accuracy than transfer learned models.
Transfer Learned NN Models	Local	3.5	Python	Aryan, Trent, Mani	Likewise to simulated models, the models can play generalized chess. Less centipawn loss than simulated base model.
Hold-Out Validation	Local	5	Python	Aryan, Mani	Fully Functional feature, done as part of the model training
Prediction gameplay against Stockfish	Local	5	Python	Aryan	This allows users to play their trained model against Stockfish for 100 games. Fully Functional

Accuracy & Centipawn Loss Evaluation	Local	5	Python	Aryan, Trent	Fully Functional feature.
Interface	Local	3.5	Python	Aryan	Showcases a game with the user's model chosen vs Stockfish AI, still can't allow our model to be played against humans.

External Libraries & Tools

Inspired Source Code

Our work was inspired in part by the implementation provided by the GitHub user Skripkon.

Source: https://github.com/Skripkon/chess-engine/blob/main/engines/tensorflow/train_and_predict.ipynb

Datasets

Stockfish AI: In-house simulated self-played games using: `python-chess` library

Lichess Elite Database: <https://database.lichess.org/>

Libraries used for our Neural Network

- TensorFlow / Keras
- python-chess
- Tqdm

Data pre-processing, Cleaning, & Evaluation

- statsmodels
- pyspark
- numpy
- pandas
- scipy
- Matplotlib

Installation Steps

1. Install Python

Our project requires Python 3.9+. <https://www.python.org/downloads/>

One-Line Installation

```
pip install tensorflow python-chess tqdm numpy pandas scipy statsmodels pyspark matplotlib
```

**Recommend to set up a virtual environment for this project for these installations.*